

Object-oriented programming in C++

Types and Operators

Dr. Claudia Weis, Faculty of Computer Science

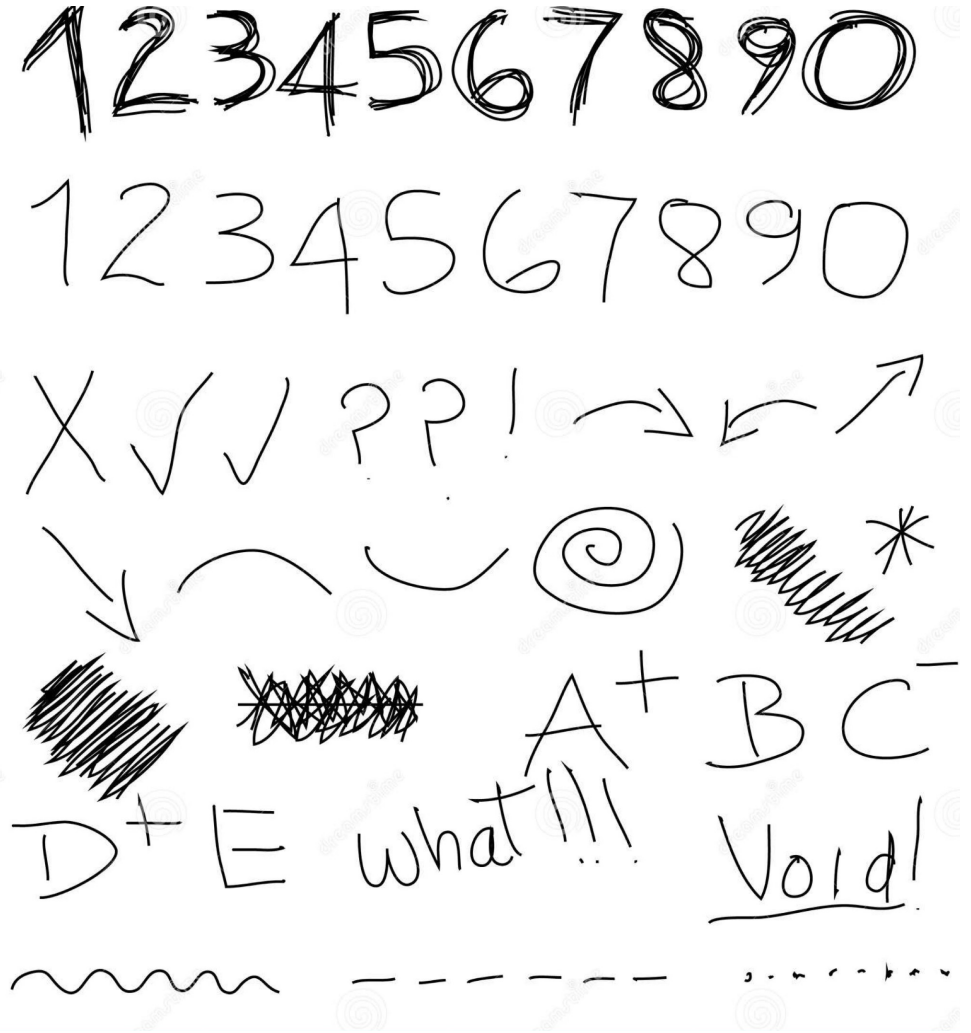
Fakultät für Informatik

UNIVERSITÄT DUISBURG ESSEN

Offen im Denken

How humans store information

- writing and drawing e.g. in giant notebooks



How do computers store information?

- as binary code



Reminder: Bases

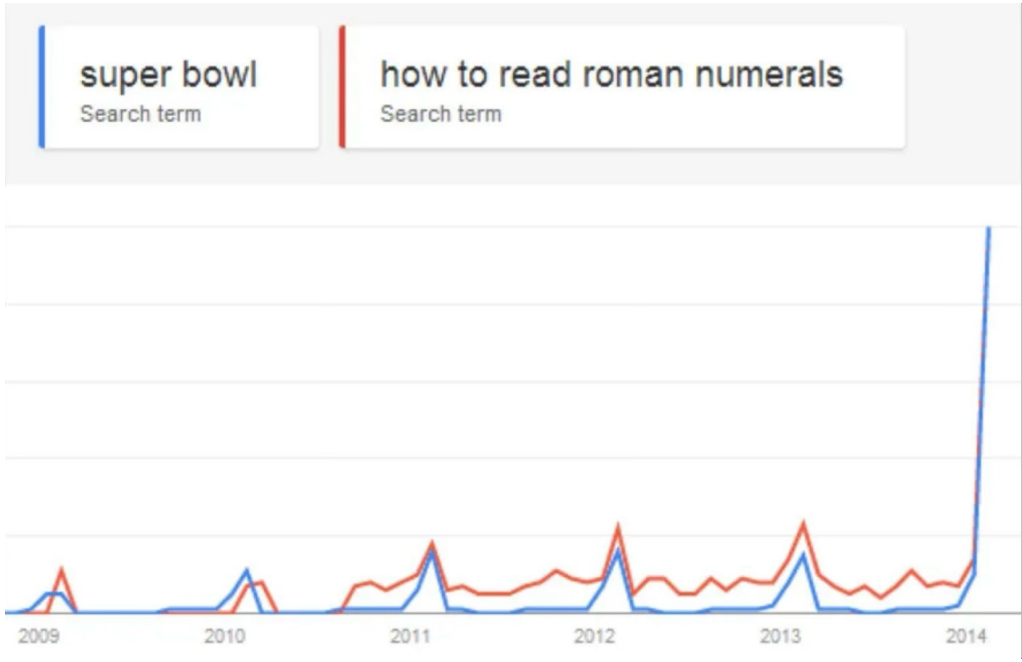
- Romans used letters for numbers

- I (1), V (5), X (10), L (50),
- C (100), D (500), M (1000)

- XL (40), VIII (8), MMXXV (2025)

- Maximum: MMMCMXCIX (3999)
- Minimum: I (1)

- Not suitable for most mathematical use cases



Reminder: Bases + positional arithmetic

- Unary (crossings on walls e.g. ııııııııııı)
- 2 (binary)
- 8 (octal)
- 10 (we)
- 12 (Egyptians)
- 16 (hexadezimal)
- 20 (Mayans)
- 60 (Babylonians, clocks, angles)

$$Number = \sum_{power=0}^{n} value \cdot base^{power}$$

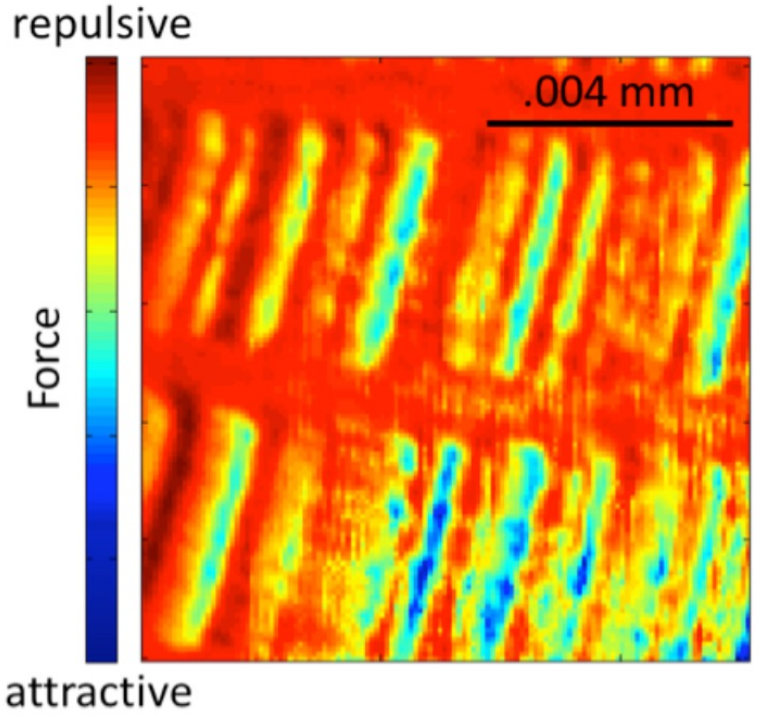
Examples: Bases + positional arithmetic

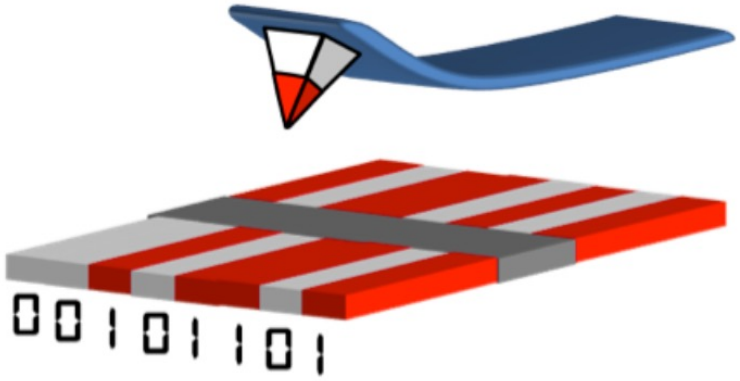
$$Number = \sum_{power=0}^{n} value \cdot base^{power}$$

- Base 10: $271 = 2 \cdot 10^2 + 7 \cdot 10^1 + 1 \cdot 10^0$
  - digits (decimal): 2 7 1
- Base 16: $271 = 1 \cdot 16^2 + 0 \cdot 16^1 + 5 \cdot 16^0$
  - digits (hexadecimal): 1 0 5
- Base 2: $271 = 1 \cdot 2^8 + 0 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$
  - digits (binary): 1 0 0 0 0 1 1 1 1

No exception: Computers store everything as either 0 or 1

- Integer Numbers
- Floating Point Numbers
- Images
- Movies
- Games
- Websites





7 is 111 in binary

- So we can store 111 just somewhere?
  - Not so easy…
- Example:

00101010*111*00101001010101001001

- How to understand that this *111* is our 7?
  - maybe we mean to store 10111?

- Fixed-width memory cells
  - Each cell is a byte, 8 bits
  - Everything uses at least one byte

7 is 0000 0111 in binary (as a byte)

- 8 bit store 0-255
  - 8 bit means 8 binary digits, i.e. the maximum is 1 1 1 1 1 1 1 1

$$\max = 1 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$$

- Use more bytes, if we need more

- 16 bit $\longrightarrow$ 65 536
- 32 bit $\longrightarrow$ 4 294 967 296
- 64 bit $\longrightarrow$ 18 446 744 073 709 551 616
- 128 bit $\longrightarrow 3.4028236692 \cdot 10^{38}$

Why only 2,4,8,16… bytes?

- Operating systems need to manage storage
- Easier to manage these few sizes than arbitrary sizes
  - First fit, next fit, best fit, etc. problems



Fundamental data types

- Integer Numbers : ... -3, -2, -1, 0, 1, 2, 3, ...
- Floating Point Numbers: 0.5, 3.141, 42.0, ...
- Boolean: true, false
- Char: 'a'
- String: "Hello Class"
- Array: [1,2,3]

Integer: stores a number

```cpp
#include <iostream>

  int main() {
      std::cout << "Lets sum all integer numbers up to 100." << std::endl;


      int sum = 0;
      for (int k = 1; k <= 100; k++) {
          sum = sum + k;
      }


      std::cout << sum << std::endl;
      return 0;
  }
```

```
> main.out
Lets sum all integer numbers up to 100.
5050

Process finished with exit code 0
```

What if you add a large number?

- Only the bits used in your data type (e.g. last 8, last 16,...) are considered
- Example: 8 bit

$$1111\ 1111 + 1 = 1\ 0000\ 0000$$

- C++ does not automatically use the next bigger data type

What's the largest number we can store?

Fakultät für Informatik

UDE

```
#include <iostream>

  int main() {
      std::cout << "Lets find the largest int." << std::endl;

      int largest = 1;
      while ( largest > 0 ) {
          largest++;
      }

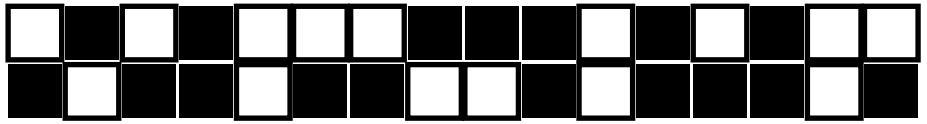
      largest--;
      std::cout << largest << std::endl;
      return 0;
  }
```

```
> main.out
Lets find the largest int.
2147483647

Process finished with exit code 0
```

The size of an int depends on your system

- An `int` is limited in what it can store.
- It allocates a fixed number of bits that can be used to store a number that fits in this size.

- What are the capacities? IT DEPENDS!
- 16 bit machines ⟶ 16 bit 
- 32+ bit machines ⟶ 32 bit 

- There are many qualifiers for integer numbers like `short`, `long` etc.

Eliminate the guesswork

```
#include <cstdint>
```

- `uint8_t / int8_t` for unsigned/signed integers of 8 bits width
- `uint16_t / int16_t` for unsigned/signed integers of 16 bits width
- `uint32_t / int32_t` for unsigned/signed integers of 32 bits width
- `uint64_t / int64_t` for unsigned/signed integers of 64 bits width

largest integer number - revisited

```cpp
#include <iostream>
#include <cstdint>

int main() {
    std::cout << "Lets find the largest int32_t." << std::endl;

    int32_t_ largest = 1;
    while ( largest > 0 ) {
        largest++;
    }

    largest--;
    std::cout << largest << std::endl;
    return 0;
}
```

```
> main.out
Lets find the largest int32_t.
2147483647

Process finished with exit code 0
```

2's complement
How to store negative numbers?

- First bit indicates the sign
  - 0 for positive numbers
  - 1 for negative numbers

- Conversion in three steps:
  (1) compute the binary representation of the absolute value of your number
  (2) flip all bits
  (3) add +1

Example: Lets store -6 using 4 bits.

(1) binary representation of 6:

0 1 1 0

because

$$6 = 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$$

(2) flip all bits:

1 0 0 1

(3) add +1

1 0 0 1 + 1 = 1 0 1 0

- remember that the first bit indicates the sign

2's complement
How to interpret binary representations?

Example:

1 0 1 0

- Remember: the first bit indicates the sign

Conversion to decimal representation:

- First bit is 1, i.e. number is negative

$$-1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = -8 + 2 = -6$$

2's complement – extra info
What about 0?

(1) binary representation (4 bits):

0 0 0 0

(2) flip all bits:

1 1 1 1

(3) add +1 (and cut off, what flows out of our 4 bits):

1 1 1 1 + 1 = 1 0 0 0 0

➢ 0 and -0 are the same

2's complement – extra info
What about -127?

(1) binary representation (8 bits) of 127:

0 1 1 1 1 1 1 1

(2) flip all bits:

1 0 0 0 0 0 0 0

(3) add +1:

1 0 0 0 0 0 0 1

➢ Convert to decimal representation

$$-1 \cdot 2^7 + 1 \cdot 2^0 = -128 + 1 = -127$$

2's complements ranges

- 8 bit unsigned (all positive)
  - range: 0 – 255

    because: $2^8 = 256$ numbers starting with 0

- 8 bit signed (positive, negative, 0)
  - range: -128 – 127

- 16 bit signed
  - range: $-2^{15} - (2^{15} - 1)$

- And so on…

2's complement
Trivia and warnings

- If you reach the boundaries of an integer, you may have overflows/underflows.
- Some of them are defined behaviour, some are undefined behaviours
- Try to avoid mixing signed/unsigned integers, as you may quickly end up doing errors
- Boolean and char are integer numbers under the hood

Operators
The good

bla.cpp

```cpp
int main(){
    int x = 4;
    int y = 5;
    std::cout << x + y << std::endl; //  9
    std::cout << x - y << std::endl; // -1
    std::cout << x * y << std::endl; // 20
    std::cout << x / y << std::endl; // 0
    std::cout << 13 % 4 << std::endl; //1
    std::cout << (x == y) << std::endl; // true
    std::cout << (x != y) << std::endl; // false
}
```

Operators
The bad and ugly

bla.cpp

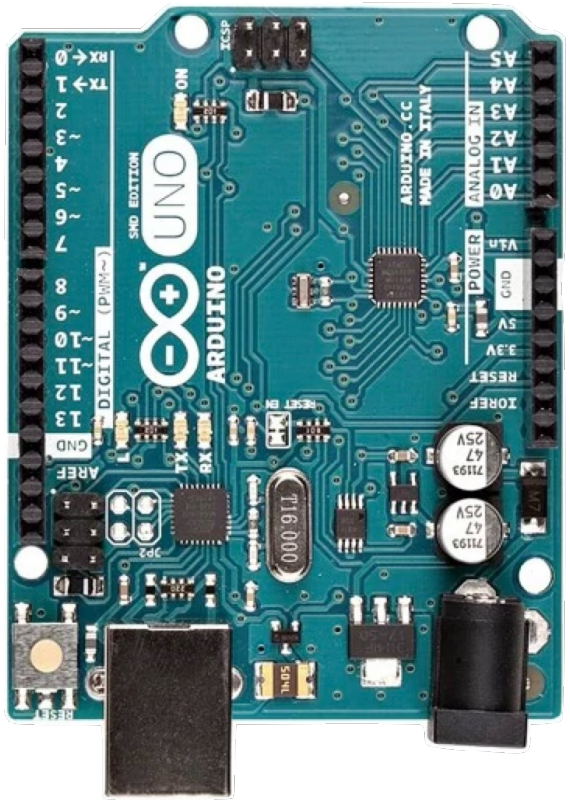
```
11  int main(){
12      std::cout << (0b101 & 0b100) << std::endl; //  0b100   Bitwise AND
13      std::cout << (0b111 ^ 0b100) << std::endl; //  0b011   Bitwise XOR
14      std::cout << (0b101 | 0b110) << std::endl; //  0b111   Bitwise OR
15      std::cout << ~(0b11111111111111111111111111111111) << std::endl; //  0b0
16      std::cout << (0b1<<3) << std::endl; //0b1000
17      std::cout << (0b1000 >> 3) << std::endl; //0b1
18  }
```

Be careful not to confuse these bitwise operations with logical operations!

Where to use these bitwise operations? Embedded example

- DDRB (Data Direction Register B)
  - Determines the direction of the signal (1 means OUTPUT, 0 means INPUT)
- PORTB (Port B Data Register)
  - Determines the state of an output (1 means ON, 0 means OFF)
- Arduino Light control, e.g. at Pin #5:
  PB5 of DDRB must be 1 (it should be an output)
  PB5 of PORTB must be 1 (it should be on)

- DDRB, PORTB are 8 bit values each, we need to set one bit!
- What about the previous state of DDRB, PORTB?
- e.g. DDRB : 00**0**11001 PORTB : 00**0**01001



Using bitwise operations

bla.cpp

```
12      DDRB  |= (1 << PB5);
13      PORTB |= (1 << PB5);
14
15      DDRB  &= ~(1 << PB5);
16      PORTB &= ~(1 << PB5);
```

Floating point numbers – float / double

bla.cpp

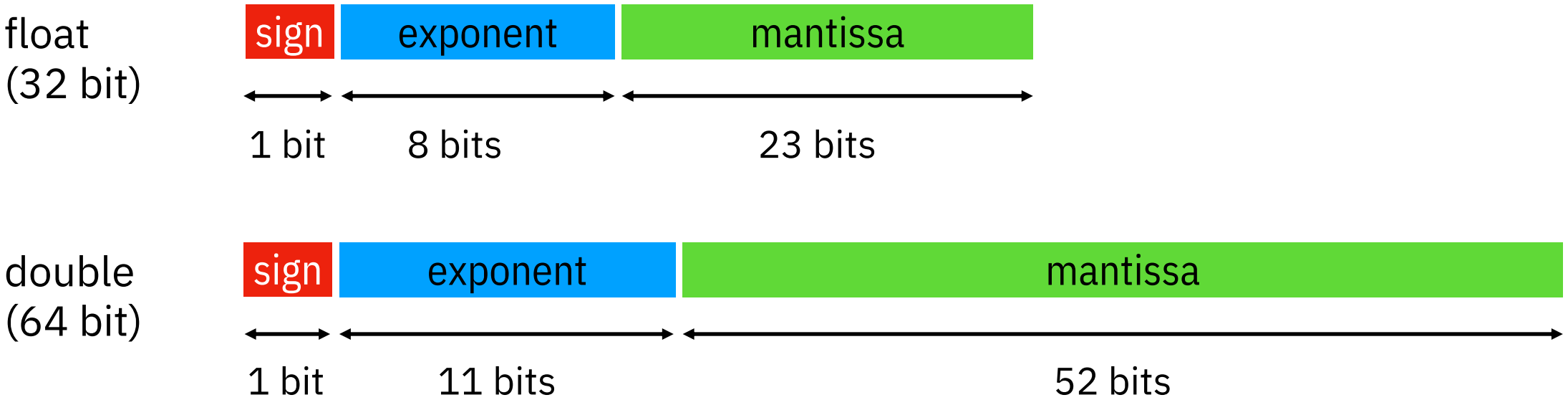
```cpp
#include <iostream>

int main()
{
    std::cout << "How much Alcohol in a beer?" << std::endl;
    float amount_beverage = 0.33;
    float alcohol_percentage = 0.047;
    float raw_alcohol = amount_beverage * alcohol_percentage;

    std::cout << raw_alcohol << std::endl;
}
```

```
> main.out
How much alcohol in a beer?
0.01551
```

IEEE 754 floating point standard



$$\pm 1.\langle mantissa \rangle \times 2^{\langle exponent \rangle}$$

Example: Euler's number

$$e = \lim_{n \to \infty} \left(1 + \frac{1}{n}\right)^n$$

$$e \approx 2.71828\ 18284\ 59045\ 23536\ 02874\ 71352\ 66249\ 77572\ 47093\ 69995$$

Float is less precise than double (1)

bla.cpp

```cpp
int main(){
    for(int i = 1; i < 30000; i++){
            float e = (1.0 + 1.0/i);
            float temp = e;
            for(int x = 0; x < i - 1; x++){
                e = e * temp;
            }
            std::cout << e << std::endl;
    }
}
```

```
2.7202
2.72029
2.72038
2.72047
2.72057
2.72066
2.72075
2.72084
2.72093
2.72102
2.72111
2.7212
2.72129
2.72138
2.72147
2.72156
2.72166
2.72175
2.72184
```

$e \approx 2.71828\ 18284\ 59045\ 23536\ 02874\ 71352\ 66249\ 77572\ 47093\ 69995$

Float is less precise than double (2)

bla.cpp

```cpp
int main(){
    for(int i = 1; i < 30000; i++){
            double e = (1.0 + 1.0/i);
            double temp = e;
            for(int x = 0; x < i - 1; x++){
                e = e * temp;
            }
            std::cout << e << std::endl;
    }
}
```

```
2.71824
2.71824
2.71824
2.71824
2.71824
2.71824
2.71824
2.71824
2.71824
2.71824
2.71824
2.71824
2.71824
2.71824
2.71824
2.71824
2.71824
2.71824
2.71824
2.71824
2.71824
```

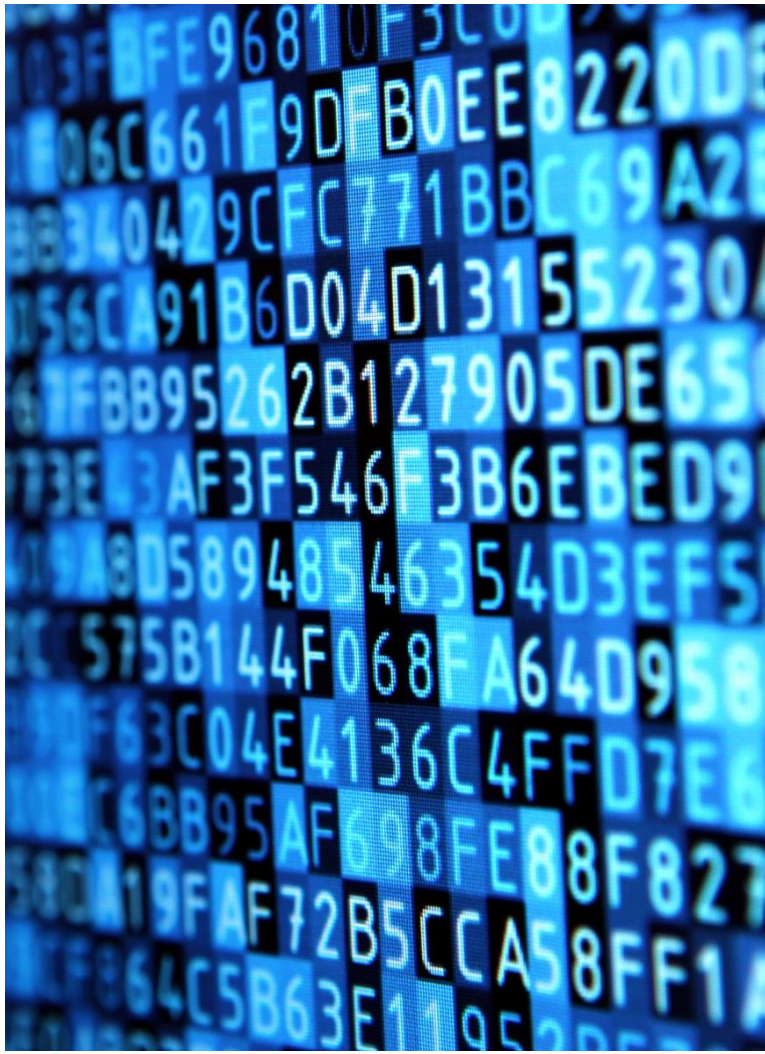
$e \approx 2.71828\ 18284\ 59045\ 23536\ 02874\ 71352\ 66249\ 77572\ 47093\ 69995$

Machine numbers

- finite set of machine numbers

$$\mathbb{M} = \{-\tilde{a}_{min}, \dots, \tilde{a}_{max}\}$$

- only real numbers with a limited fix number n_d of digits
- We need to map each real number to a machine number.
- all real numbers with the same leading n_d digits are mapped to the same machine number
- machine numbers are
  - not spread regularly on the number line,
  - dense near zero
  - sparse at the end of the computation range $[-\tilde{a}_{min}, \tilde{a}_{max}]$



Be careful!

- Floating point numbers work differently than our intuition suggests
- In general, we cannot represent all of them fully without truncation
- “Clean“ numbers in base 10 may be periodic numbers in base 2

bla.cpp

```cpp
#include <iostream>

int main()
{
    float num = 0.00;
    while(num != 8.0){
        num += 0.1;
        std::cout<< num << std::endl;
    }
}
```

```
7.3
7.4
7.5
7.6
7.7
7.79999
7.89999
7.99999
8.09999
8.2
8.3
8.4
```

NEVER compare floating point numbers using == or !=

Bool

- Boolean variables have one of two values
  - true or false
  - 1 or 0
  - in C++ also possible: any other number, only 0 is mapped to false



Boolean relations and operators

notion	relation / operation
==	equal
!=	unequal
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
!	negation (NOT)
&&	logical conjunction (AND)
||	logical disjunction (OR)

- Mind the difference:
  - == comparison of two quantities
  - = assignment of a value to a variable

```
#include <iostream>

int main() {
    bool a = true;
    bool b = false;

    std::cout << (3 == 5) << std::endl;
    std::cout << (3 < 5) << std::endl;

    bool a_and_b = a && b;
    std::cout << a_and_b << std::endl;)

    bool not_b = !b;
    std::cout << not_b << std::endl;

    return 0;
}
```

Char
stores a character

bla.cpp

```cpp
#include <iostream>

int main()
{
    char a = 'a';
    std::cout << a << std::endl;
    char c = 'c';
    c++;
    std::cout << c << std::endl;
}
```

```
❯ ./a.out
a
d
```

ASCII	Hex	Symbol
0	0	NUL
1	1	SOH
2	2	STX
3	3	ETX
4	4	EOT
5	5	ENQ
6	6	ACK
7	7	BEL
8	8	BS
9	9	TAB
10	A	LF
11	B	VT
12	C	FF
13	D	CR
14	E	SO
15	F	SI

ASCII	Hex	Symbol
16	10	DLE
17	11	DC1
18	12	DC2
19	13	DC3
20	14	DC4
21	15	NAK
22	16	SYN
23	17	ETB
24	18	CAN
25	19	EM
26	1A	SUB
27	1B	ESC
28	1C	FS
29	1D	GS
30	1E	RS
31	1F	US

ASCII	Hex	Symbol
32	20	(space)
33	21	!
34	22	"
35	23	#
36	24	$
37	25	%
38	26	&
39	27	'
40	28	(
41	29	)
42	2A	*
43	2B	+
44	2C	,
45	2D	-
46	2E	.
47	2F	/

ASCII	Hex	Symbol
48	30	0
49	31	1
50	32	2
51	33	3
52	34	4
53	35	5
54	36	6
55	37	7
56	38	8
57	39	9
58	3A	:
59	3B	;
60	3C	<
61	3D	=
62	3E	>
63	3F	?

ASCII	Hex	Symbol
64	40	@
65	41	A
66	42	B
67	43	C
68	44	D
69	45	E
70	46	F
71	47	G
72	48	H
73	49	I
74	4A	J
75	4B	K
76	4C	L
77	4D	M
78	4E	N
79	4F	O

ASCII	Hex	Symbol
80	50	P
81	51	Q
82	52	R
83	53	S
84	54	T
85	55	U
86	56	V
87	57	W
88	58	X
89	59	Y
90	5A	Z
91	5B	[
92	5C	\
93	5D	]
94	5E	^
95	5F	_

ASCII	Hex	Symbol
96	60	`
97	61	a
98	62	b
99	63	c
100	64	d
101	65	e
102	66	f
103	67	g
104	68	h
105	69	i
106	6A	j
107	6B	k
108	6C	l
109	6D	m
110	6E	n
111	6F	o

ASCII	Hex	Symbol
112	70	p
113	71	q
114	72	r
115	73	s
116	74	t
117	75	u
118	76	v
119	77	w
120	78	x
121	79	y
122	7A	z
123	7B	{
124	7C	|
125	7D	}
126	7E	~
127	7F	□

Fakultät für Informatik

UDE

String
Surprisingly difficult topic!

- Store several characters
- Not a fundamental data type, however implemented with fundamental types

Example:

bla.cpp

```cpp
#include <iostream>
#include <string>
int main()
{
    std::string s = "Hello World.";
    std::cout << s;
}
```

```
❯ ./a.out
Hello World.
```

Extending strings (1)

bla.cpp

```
1  #include <iostream>
2  #include <string>
3  int main()
4  {
5      std::string s = "Hello ";
6      s += "World.";
7      std::cout << s;
8  }
```

- Line 5: creation of a string object with name s
- Line 6: extension of the string object with another string using the += operator
- What happens: no new string is created, the memory of s is simply extended

Extending strings (2)

bla.cpp

```
1  #include <iostream>
2  #include <string>
3  int main()
4  {
5      std::string s = "Hello";
6      char c = '!';
7      s += c;
8      std::cout << s;
9  }
```

- Line 5: creation of a string object with name s
- Line 6: extension of the string object with a single character using the += operator
- What happens: the memory of s is extended such that it can hold the extended string.

Concatenating strings

bla.cpp

```
1  #include <iostream>
2  #include <string>
3  int main()
4  {
5      std::string s1 = "Hello ";
6      std::string s2 = "World.";
7      std::string s3 = s1 + s2;
8      std::cout << s3;
9  }
```

- Lines 5 and 6: creation of two separate strings `s1` and `s2`
- Line 7: `s1` and `s2` are joined together into a completely new string `s3` using the + operator.
  - New memory is allocated to hold `s3` .
  - The original variables `s1` and `s2` remain unchanged.

Indexing strings – extracting single characters

```cpp
#include <iostream>

#include <string>

   int main() {

       std::string s = "Hello World!";

       char c1 = s[0];
       std::cout << "First character: " << c1 << std::endl;

       char c2 = s.at(0);
       std::cout << "First character again: "<< c2 << std::endl;

       char c3 = s[6];
       std::cout << "7th character: " << c3 << std::endl;

       char c4 = s.at(6);
       std::cout << "7th character again: "<< c4 << std::endl;

       return 0;
   }
```

```
> main.out
First character: H
First character again: H
7th character: W
7th character again: W

Process finished with exit code 0
```

Extracting single characters – What can go wrong?

```cpp
#include <iostream>
#include <string>

   int main() {

       std::string s = "Hello World!";
       char c5 = s[42];
       std::cout << "Out of boundary character: " << c5 << std::endl;

       char c6 = s.at(42);
       std::cout << "Out of boundary character again: "<< c6 << std::endl;

       return 0;
```

unsafe old (but very common) style

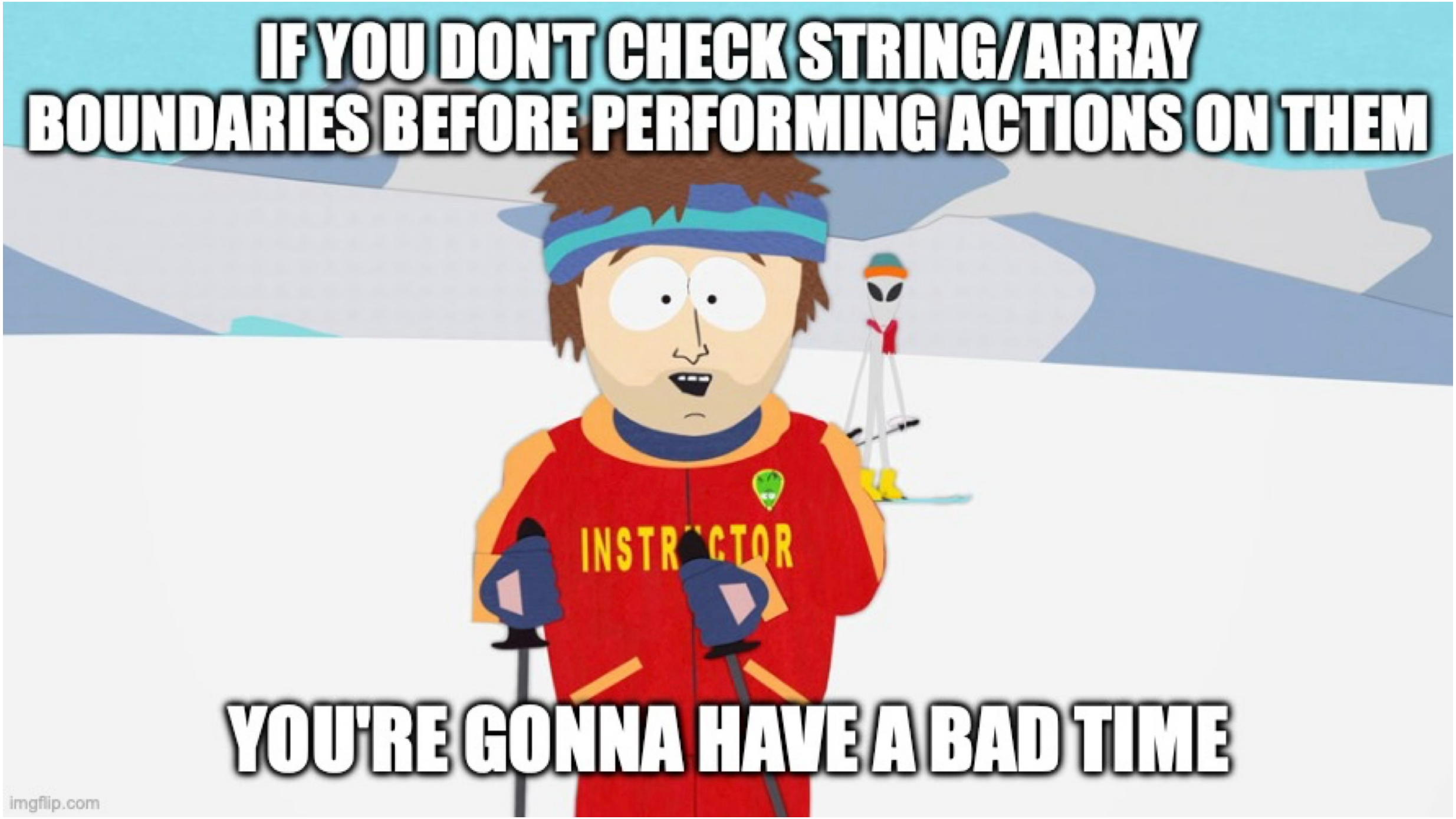
safe, modern style with exception handling

```
> main.out
Out of boundary character:
libc++abi: terminating due to uncaught exception of type std::out_of_range: basic_string

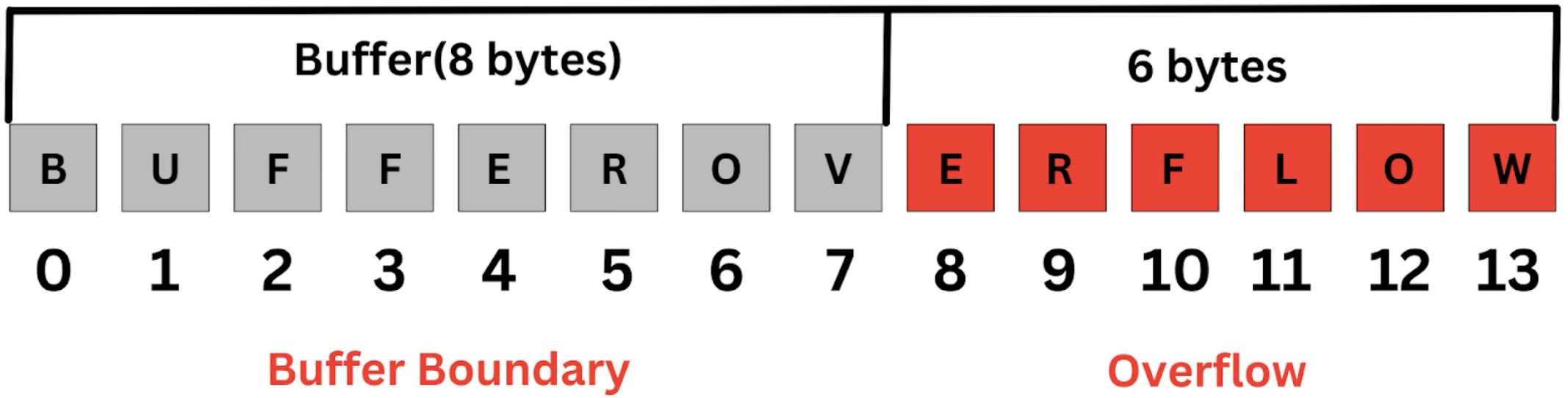
Process finished with exit code 134 (interrupted by signal 6:SIGABRT)
```

String
Trivia and warnings

- If you need to support UTF-8 in your strings, you will need more help than this slide can offer
- Note: UTF-8 has variable length code points.
  - 16 Bytes of UTF-8 can be anything between 4 and 16 code points
- *Some* strings are null terminated ('\0' at the end) to indicate the end.


IF YOU DON'T CHECK STRING/ARRAY
BOUNDARIES BEFORE PERFORMING ACTIONS ON THEM
INSTRUCTOR
YOU'RE GONNA HAVE A BAD TIME
imgflip.com

Buffer and buffer overflow

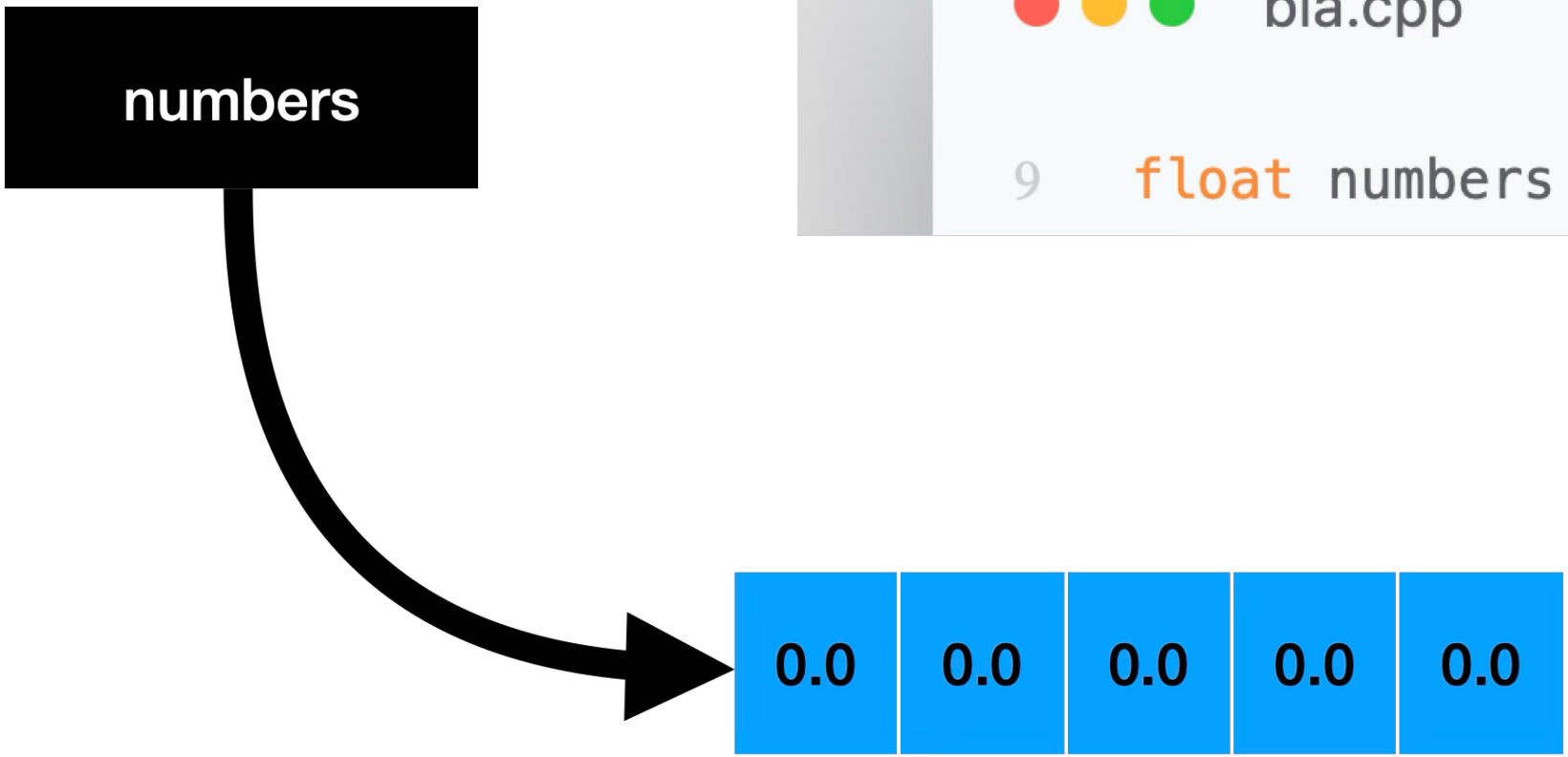


- Buffer: reserved memory of fixed size, here: 8 bytes
- Buffer overflow: one of the most well-known and dangerous vulnerabilities in software development
- especially in C/C++

Array
- understandable explanation -

- Fixed-sized list of elements of a single type
- You cannot resize the array
- All elements are stored in succession
- All elements are of the same type

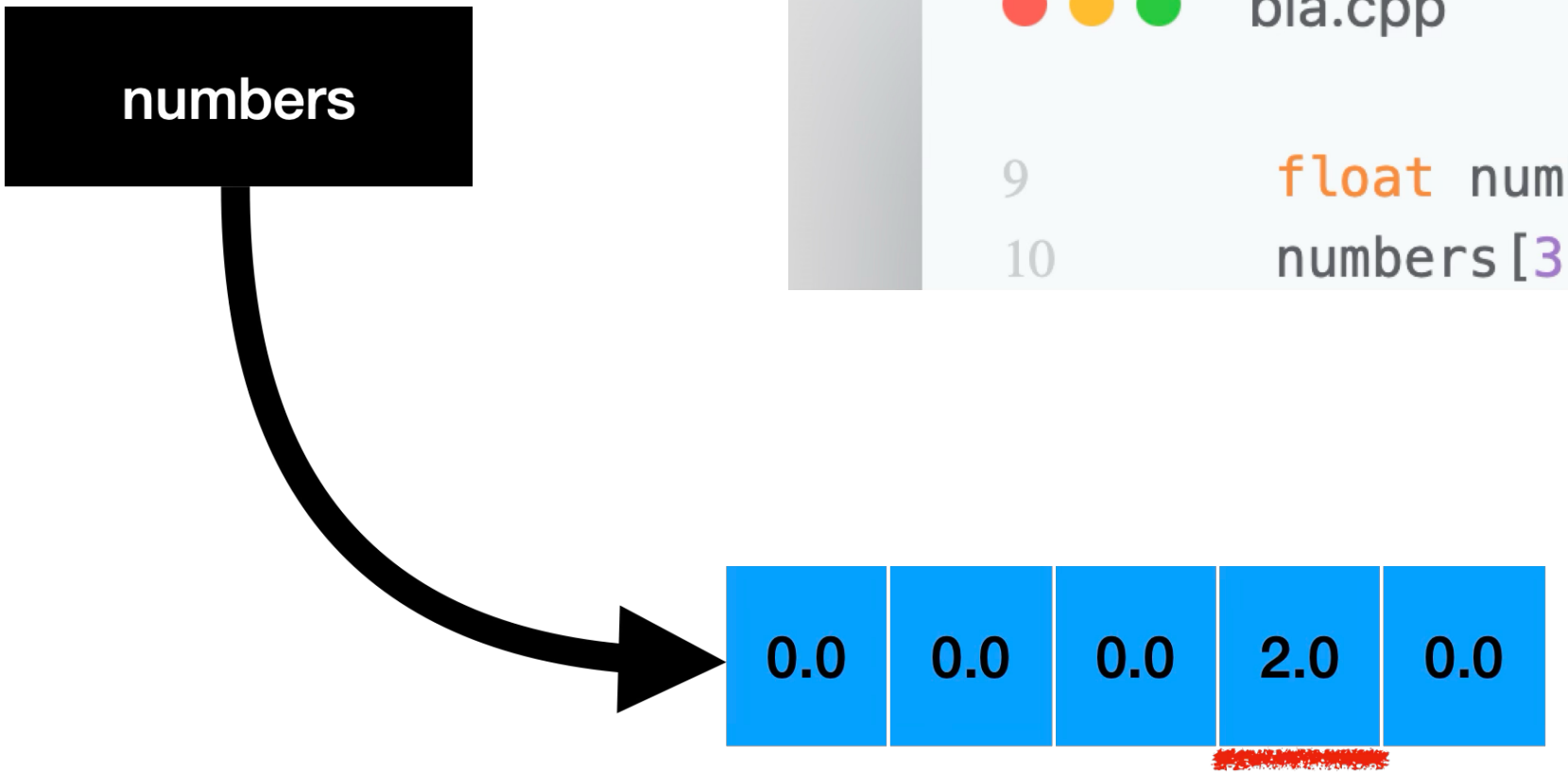
Creating an array




```
bla.cpp

9   float numbers[5];
```

Changing elements in an array




```
bla.cpp

9       float numbers[5];
10      numbers[3] = 2.0;
```

Array
Trivia and warnings

- A reserved memory region of `sizeof(element) * number`
- The variable that refers to an array is technically not an array, but a pointer to the start of the reserved region.
- The size of that region is not stored inline, if you need it, you need to keep track of it!

What is numbers[5] *?*

- Accessing elements in an array with the brackets [ ] is syntactic sugar.

```
numbers[5]
*(numbers + 5)
*(5 + numbers)
5[numbers]
```

- Are all the same



Entertaining for experienced C++ programmers, confusing (or scary?) for beginners…
⟶ Don't panic ;-) We'll talk about pointers in more detail later…

Array
infinite uses - one example

- Small coding exercise
- Sort an array
- Array is filled with random numbers between 0 and 100

Create randomly sorted array

bla.cpp

```cpp
#include <iostream>
#include <ctime>
#include <cstdlib>

int main()
{
    int numbers[100];
    srand((unsigned)time(0)); //seed RNG
    for(int i = 0; i < 100; i++){
        numbers[i] = rand()%100;
        std::cout << numbers[i] << ' ' ;
    }
    std::cout << std::endl;
}
```

Check if it's actually random(ish)

```
❯ ./a.out
97 63 43 14 42 52 22 79 3 56 59 88 72 15 72 26 93 50 36 14 51 23 18 27 82 84 57 52 5 2 71 38 70 55 0 33 52 96 62 30 7 87 59 13 98 1
0 54 30 53 64 34 88 96 42 50 60 20 14 21 91 57 76 34 49 67 35 30 20 45 16 91 52 40 66 28 35 58 42 18 13 32 66 23 86 30 58 78 29 33
78 82 7 56 85 20 25 20 44 70 54
❯ ./a.out
11 61 95 83 55 96 10 88 55 80 92 71 62 99 99 85 53 8 16 38 10 14 89 47 55 3 55 14 90 78 71 96 47 89 26 52 63 0 26 25 94 37 98 38 74
 0 63 18 41 31 89 47 12 83 86 40 21 59 87 97 7 61 85 50 75 94 84 36 7 82 40 67 10 98 11 16 90 14 74 57 68 54 81 94 70 1 41 43 41 80
 11 6 19 2 77 63 5 24 99 42
❯ ./a.out
18 10 21 94 85 68 54 19 78 42 32 89 7 94 86 41 80 37 9 50 13 36 98 57 68 36 54 92 59 13 21 75 59 56 89 85 95 49 58 46 61 62 44 27 5
9 45 44 9 88 88 43 0 73 27 7 83 98 8 70 0 29 27 34 74 29 0 61 44 88 15 91 48 98 64 29 30 6 50 52 79 36 48 10 95 40 96 99 0 45 31 99
 29 24 84 82 82 21 61 37 39
❯ ./a.out
18 10 21 94 85 68 54 19 78 42 32 89 7 94 86 41 80 37 9 50 13 36 98 57 68 36 54 92 59 13 21 75 59 56 89 85 95 49 58 46 61 62 44 27 5
9 45 44 9 88 88 43 0 73 27 7 83 98 8 70 0 29 27 34 74 29 0 61 44 88 15 91 48 98 64 29 30 6 50 52 79 36 48 10 95 40 96 99 0 45 31 99
 29 24 84 82 82 21 61 37 39
❯ ./a.out
25 59 94 52 68 40 98 97 54 4 25 54 52 36 73 44 60 66 49 62 69 58 60 67 81 22 53 23 75 1 71 1 71 23 99 71 27 51 90 67 28 87 37 16 97
 90 72 53 82 45 97 6 81 71 75 73 22 57 53 3 4 46 36 51 83 53 91 52 69 48 89 29 33 30 94 97 22 86 77 48 4 42 39 49 10 44 57 4 49 82
87 52 82 19 34 1 90 51 75 83
```

Sort

bla.cpp

if we know the length of the array, everything is fine

```
18      for(int i = 0; i < length - 1; i++ ){
19          for(int j = i+1; j < length; j++ ){
20              if(numbers[j] < numbers[i] ){
21                  std::swap(numbers[i], numbers[j]);
22              }
23          }
24      }
```

```
❯ ./a.out
91 84 39 47 19 30 95 59 26 5 63 64 39 86 63 63 19 33 87 26 55 37 3 86 76 75 12 43 55 12 74 72 0 47 67 53 49 84 41 6 23 7 88 49 30 6
7 56 39 45 66 71 41 69 87 88 10 20 4 10 66 51 17 32 26 64 65 96 53 63 22 23 78 60 47 69 79 34 85 57 65 59 65 65 97 7 21 35 79 1 40
9 71 36 63 64 24 64 66 4 27
0 1 3 4 4 5 6 7 7 9 10 10 12 12 17 19 19 20 21 22 23 23 24 26 26 26 27 30 30 32 33 34 35 36 37 39 39 39 40 41 41 43 45 47 47 47 49
49 51 53 53 55 55 56 57 59 59 60 63 63 63 63 63 64 64 64 64 65 65 65 65 66 66 66 67 67 69 69 71 71 72 74 75 76 78 79 79 84 84 85 86
 86 87 87 88 88 91 95 96 97
```

But what, if we do not know about the length?

Fakultät für Informatik

UDE

```
sizeof(x)
```

⟶ how many bytes does x take?

Example

```
13  int main()
14  {
15      float numbers[5] = {0.0, 1.0, 2.0, 3.0, 4.0};
16      for(int i = 0; i < sizeof(numbers)/sizeof(float); i++){
17          std::cout << numbers[i] << std::endl;
18      }
19
20  }
```

What can go wrong?

Fakultät für Informatik

UDE

bla.cpp

```
7   void bla(float x[]){
8       for(int i = 0; i < sizeof(x)/sizeof(float); i++){
9           std::cout << x[i] << std::endl;
10      }
11  }
12
13  int main()
14  {
15      float numbers[5] = {0.0, 1.0, 2.0, 3.0, 4.0};
16      for(int i = 0; i < sizeof(numbers)/sizeof(float); i++){
17          std::cout << numbers[i] << std::endl;
18      }
19      bla(numbers);
20  }
```

Be careful, when you want to use an array as a parameter for a function!

Fakultät für Informatik

UDE

Be extremely careful,
when you want to use an array as a parameter for a function!

Understand, what's happening!

Care about compiler warnings ;-)

```
❯ g++ bla.cpp
bla.cpp:8:30: warning: sizeof on array function parameter will return size of 'float *' instead of 'float[]' [-Wsizeof-array-argument]
    8 |     for(int i = 0; i < sizeof(x)/sizeof(float); i++){
      |                              ^
bla.cpp:7:16: note: declared here
    7 | void bla(float x[]){
      |                ^
bla.cpp:8:33: warning: 'sizeof (x)' will return the size of the pointer, not the array itself [-Wsizeof-pointer-div]
    8 |     for(int i = 0; i < sizeof(x)/sizeof(float); i++){
      |                             ~~~~~~~~~~^
bla.cpp:7:16: note: pointer 'x' declared here
    7 | void bla(float x[]){
      |                ^
2 warnings generated.
❯ ./a.out
0
1
2
3
4
0
1
```

Care about compiler warnings, even if your code compiles an runs!
Very likely, you have introduced a bug that will cause unexpected behaviour.

Why our little example behaves as it behaves

- In C, array arguments in functions do not really exist.
- The compiler silently converts them into a pointer.
- Using sizeof(x) fails to determine the length of the passed array.
- Pointer Decay Effect: Because C converts array arguments into pointers, `sizeof` evaluates the pointer itself.
- Resulting error: The expression yields only the size of a memory address instead of the size of the array.

How can we do better?

bla.cpp

```
7   void bla(float *x, int length){
8       for(int i = 0; i < length; i++){
9           std::cout << x[i] << std::endl;
10      }
11  }
12
13  int main()
14  {
15      float numbers[5] = {0.0, 1.0, 2.0, 3.0, 4.0};
16      int length = sizeof(numbers)/sizeof(float);
17      for(int i = 0; i < length; i++){
18          std::cout << numbers[i] << std::endl;
19      }
20      bla(numbers, length);
21  }
```

Show that you have understood arrays (in C) and use directly a pointer. Keep track of the size and hand it over as an additional parameter.

We do not need to change numbers.

Famous wise words

You copied that function without understanding why it does what it does and as a result your code IS GARBAGE.

AGAIN.

- Linus Torvalds



Long post addressing the array issue on the Linux Kernel Mailing List (LKML) in 2015
https://lkml.org/lkml/2015/9/3/428

Fakultät für Informatik

Questions?



"live" in lecture and exercise



discussion forum @ Moodle (preferred, everyone can reply)



by e-mail (use your @uni-due.de email address!)